```
            #6
    this.partitions[elem] =
p2                                                  #7
  return true
```

#1 Takes two elements, merges their partitions and returns `true` iff the two elements were in two different partitions that now are merged, `false` if they were already in the same partition;
#2-#3 Retrieve the partitions to which `elem1` and `elem2` belong to. If the argument is invalid or not found, this call will throw.
#4 Compare `p1` and `p2`, and if they are the same there is nothing left to do, the elements are already in the same partition, and so no merge happens: `false` is returned.
#5 Loop over the elements in the first partition. For each element in `p1`:
#6 … add it to `p2`…
#7 … then update the mapping for that element, that now belongs to `p2`.

This method is more complex than the previous ones. And yet, by reusing `findPartition`, it still looks quite simple.

We first check if elements belongs to the same partition by calling `findPartition` on both and checking the result. Those calls also take care of validating the input.

Once established that we actually need to perform an action, we proceed and merge the two sets, correcting the pointers in the partitions map, when needed. If the partitions were implemented with linked lists instead of Set, we could have